



University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 02

Experiment Title: Design and Simulation of a Full Adder Using NAND-Based Structural VHDL in Xilinx ISE

Submitted by:

Name: Rifah Tasfia

Student ID: 22201126

Section: C1

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 30/09/2026

1. Experiment Name

Design and Simulation of a Full Adder Using NAND-Based Structural VHDL in Xilinx ISE

2. Objective

The main goals of this experiment are:

- To learn how basic digital logic gates work and how they are designed.
- To implement a NAND gate using VHDL.
- To build other logic gates by using NAND gate components.
- To create an XOR gate using NAND gates.
- To design a Half Adder using XOR and AND gates.
- To design a Full Adder using two Half Adders and one OR gate.
- To understand structural modeling in VHDL.
- To simulate the designed circuits using Xilinx ISE.
- To compare the simulation outputs with the related truth tables.
- To understand hierarchical circuit design, where a complex circuit is made from smaller reusable components.

3. Apparatus / Hardware & Software Requirements

- List all required tools and components:

- Xilinx ISE 14.7
- VHDL
- ISim Simulator
- Computer system for circuit design and simulation

4. Introduction

A Full Adder is a basic combinational digital circuit that adds three binary inputs. It has three inputs:

- A – First binary input
- B – Second binary input
- Cin – Carry input

It has two outputs:

- Sum – The result of the binary addition
- Cout – The carry produced by the addition

The Boolean expressions of a Full Adder are:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + (A \oplus B)\text{Cin}$$

In this experiment, the Full Adder was designed using a hierarchical structural method. First, a NAND gate was created. Then NAND gates were used to build other basic gates. After that, a Half Adder was made using XOR and AND gates. Finally, the Full Adder was built using two Half Adders and an OR gate.

The design follows this hierarchy:

NAND Gate → Basic Logic Gates → Half Adder → Full Adder

5. NAND Gate Design

A NAND gate is a universal logic gate. A two-input NAND gate gives the opposite result of an AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NAND gate was implemented using VHDL Dataflow modeling.

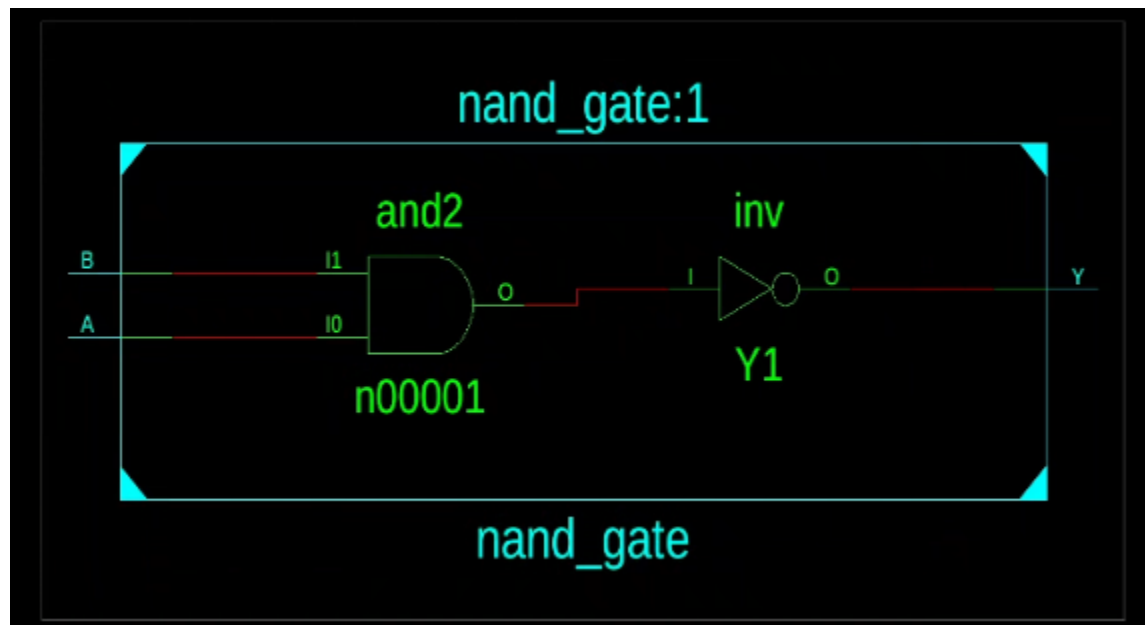


Figure 1: RTL schematic of the NAND gate.

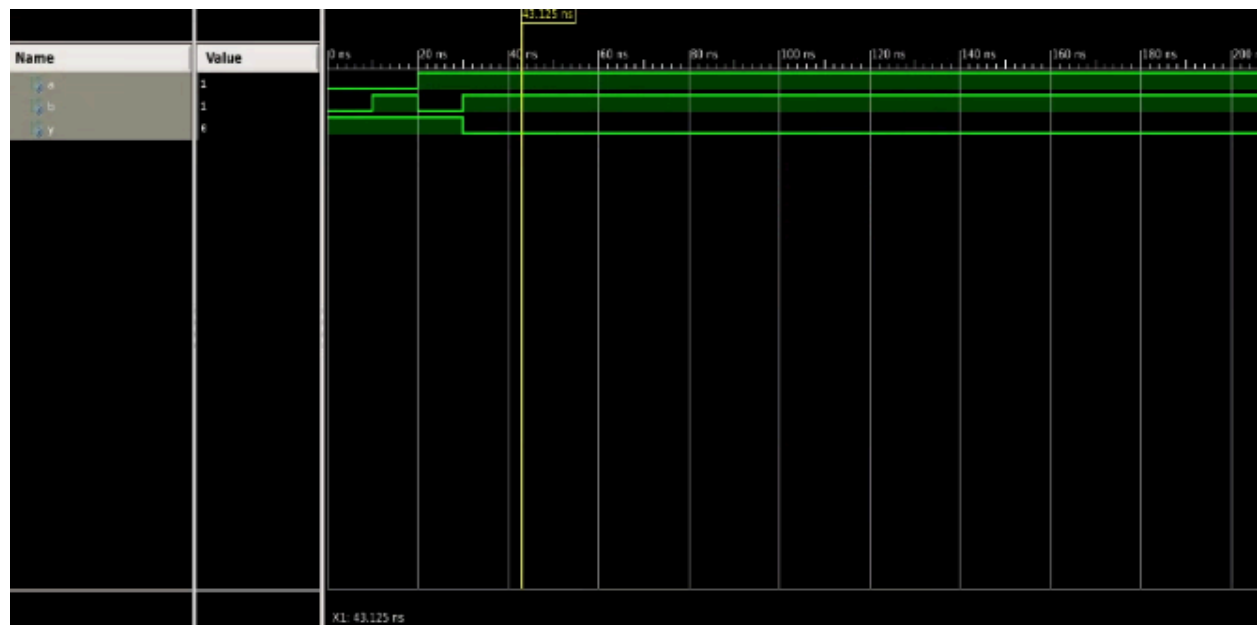


Figure 2: Simulation waveform of the NAND gate.

6. Construction of Basic Gates Using NAND Gates

NAND is a universal gate, so other logic gates can also be made by using NAND gates.

The following gates were designed using NAND gate components:

- AND
- OR
- NOR
- XOR

These gates were implemented using structural VHDL modeling.

6.1 AND Gate Using NAND Gates

An AND gate can be made using two NAND gates.

First: $N1 = A \text{ NAND } B$

Then: $Y = N1 \text{ NAND } N1$

Therefore: $Y = A \cdot B$

Number of NAND gates: 2 NAND gates

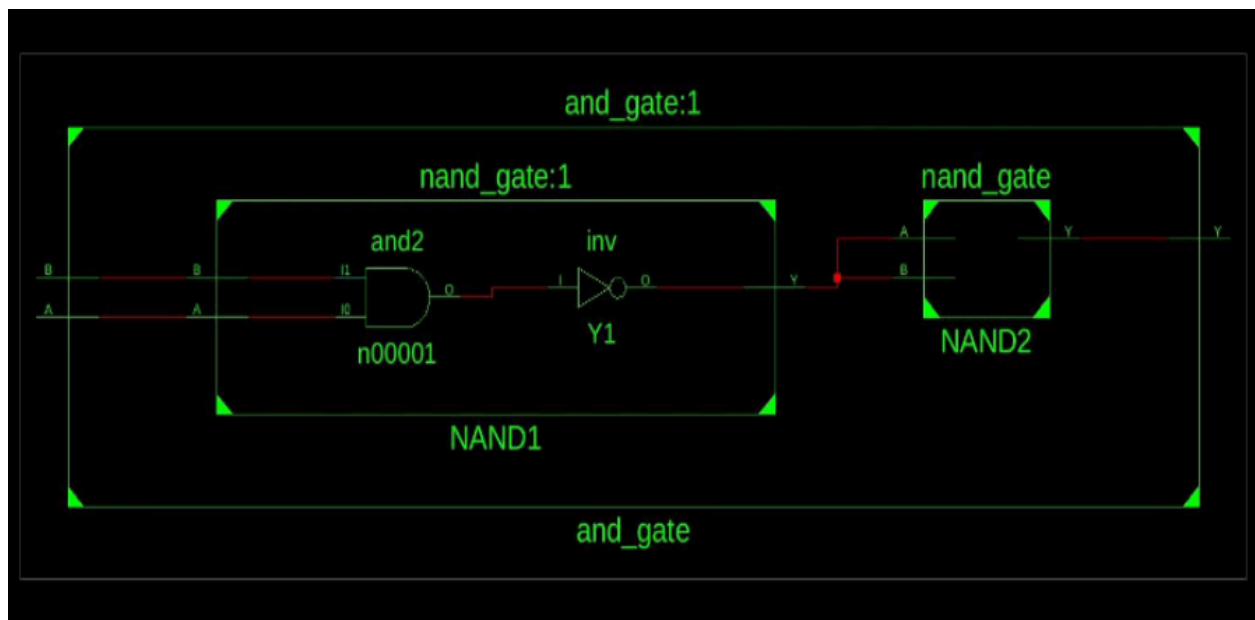


Figure 3: RTL schematic of the NAND-based AND gate.

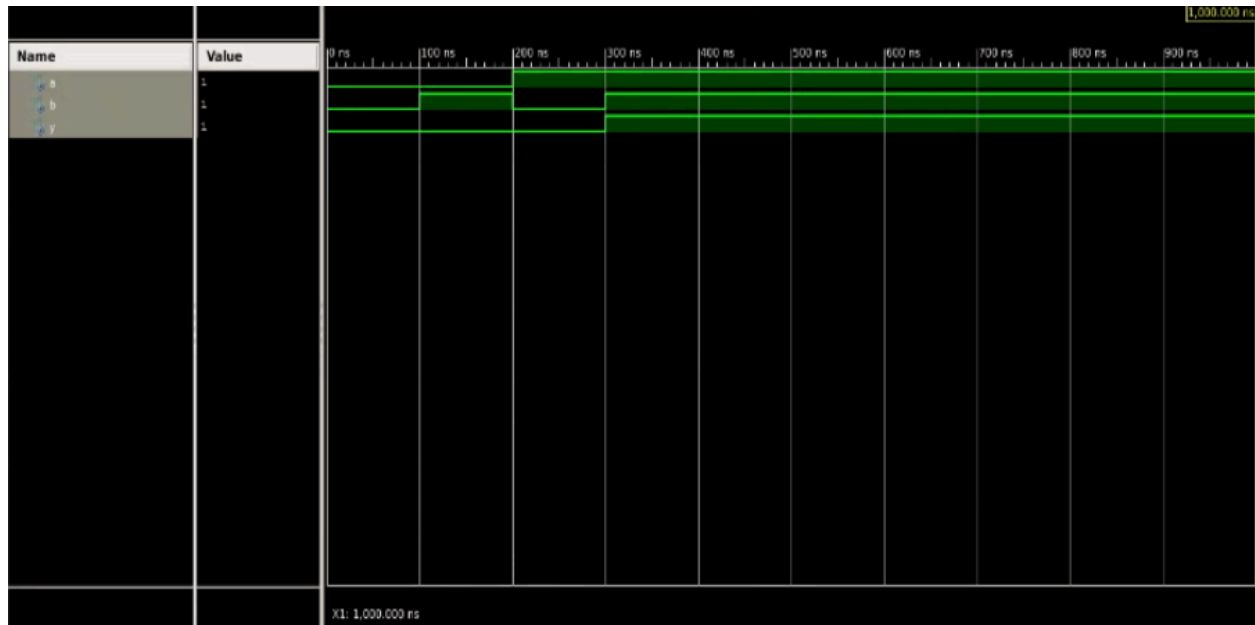


Figure 4: Simulation waveform of the AND gate.

6.2 OR Gate Using NAND Gates

Using De Morgan's theorem:

$$A + B = (\bar{A} \cdot \bar{B})$$

Therefore:

$$N1 = A \text{ NAND } A$$

$$N2 = B \text{ NAND } B$$

$$Y = N1 \text{ NAND } N2$$

So, an OR gate needs 3 NAND gates.

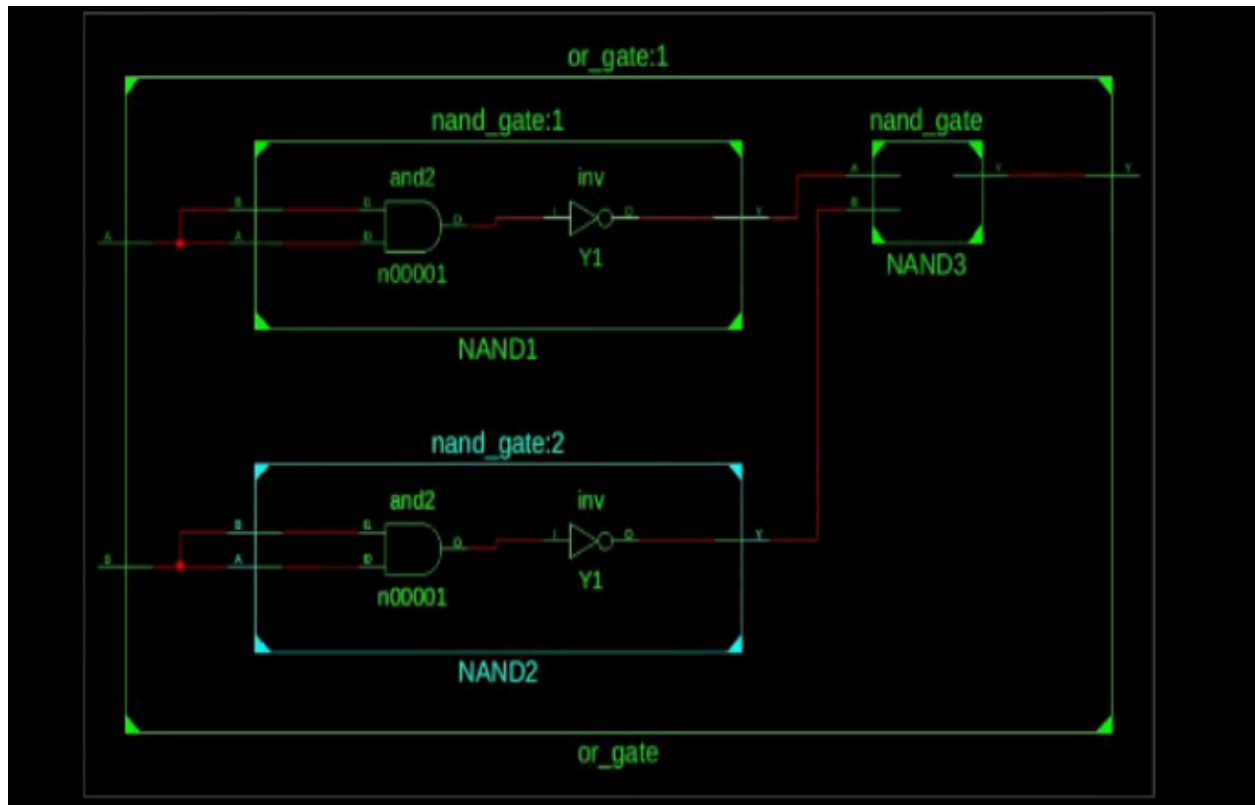


Figure 5: RTL schematic of the NAND-based OR gate.

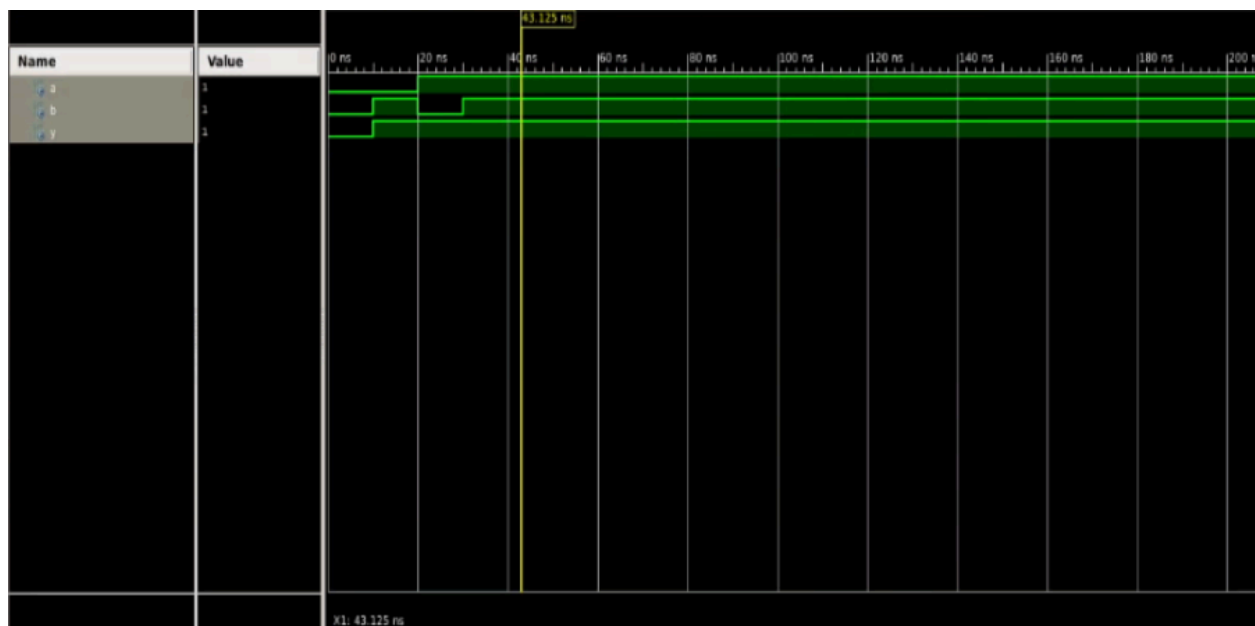


Figure 6: Simulation waveform of the OR gate.

6.3 NOR Gate Using NAND Gates

A NOR gate can be made by first creating an OR operation and then inverting its output.

This design uses four NAND gates:

$N1 = A \text{ NAND } A$

$N2 = B \text{ NAND } B$

$N3 = N1 \text{ NAND } N2$

$Y = N3 \text{ NAND } N3$

Therefore: $Y = (A + B)'$

Number of NAND gates: 4 NAND gates

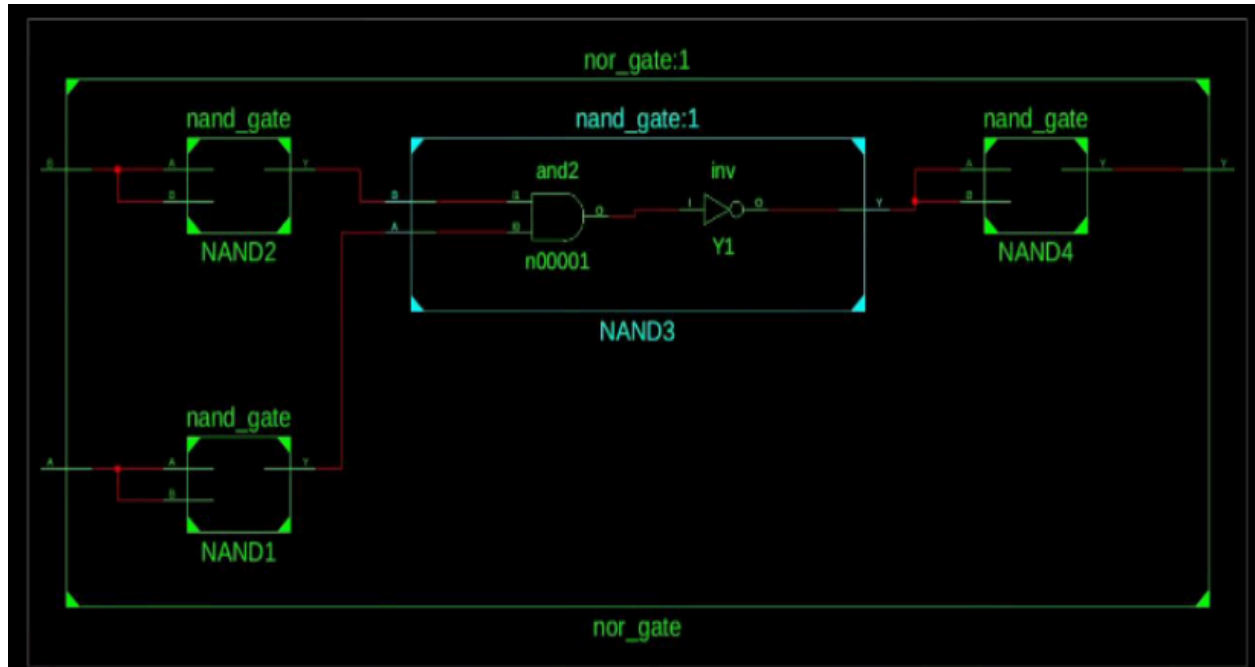


Figure 7: RTL schematic of the NAND-based NOR gate.

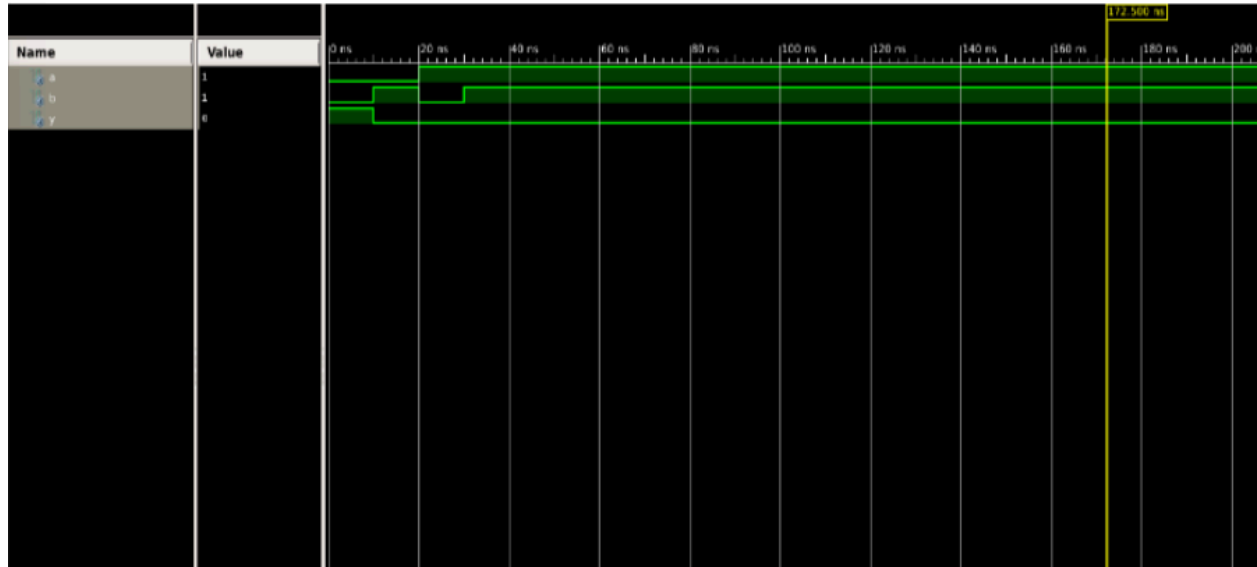


Figure 8: Simulation waveform of the NOR gate.

6.4 XOR Gate Using NAND Gates

An XOR gate gives logic 1 when its two inputs are different.

The NAND-only design uses four NAND gates:

$$N1 = A \text{ NAND } B$$

$$N2 = A \text{ NAND } N1$$

$$N3 = B \text{ NAND } N1$$

$$Y = N2 \text{ NAND } N3$$

$$\text{Therefore: } Y = A \oplus B$$

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

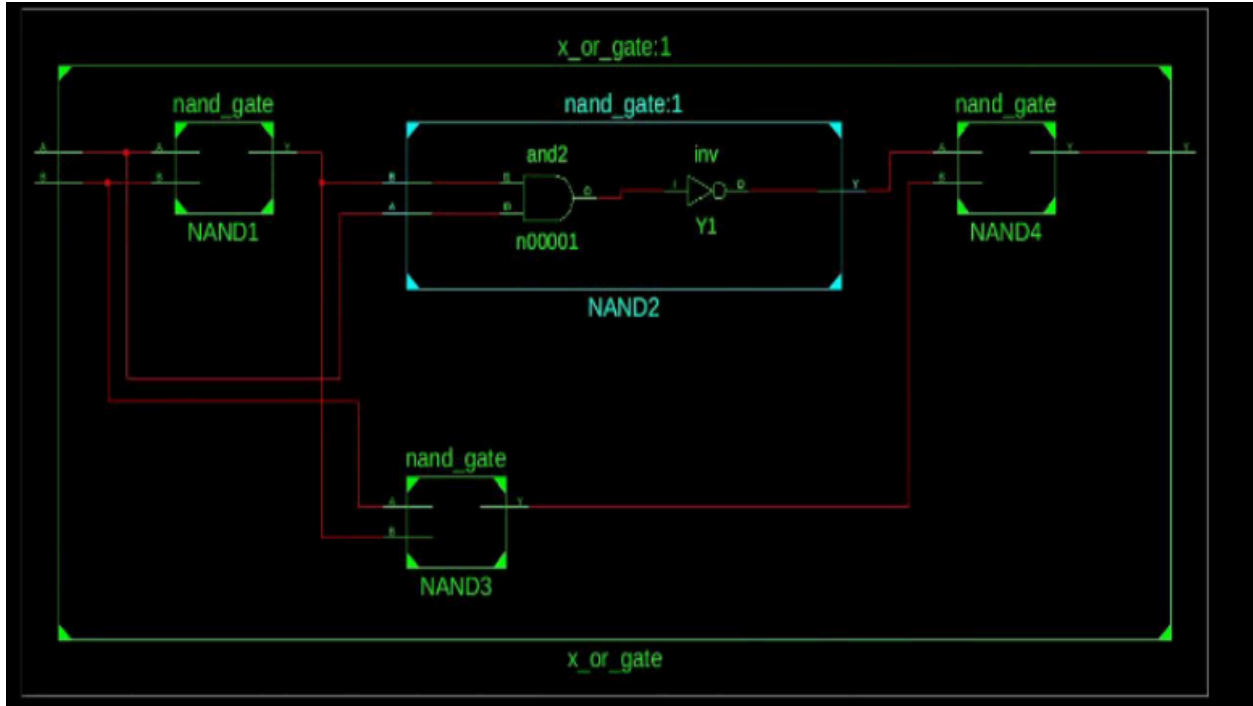


Figure 9: RTL schematic of the NAND-based XOR gate.

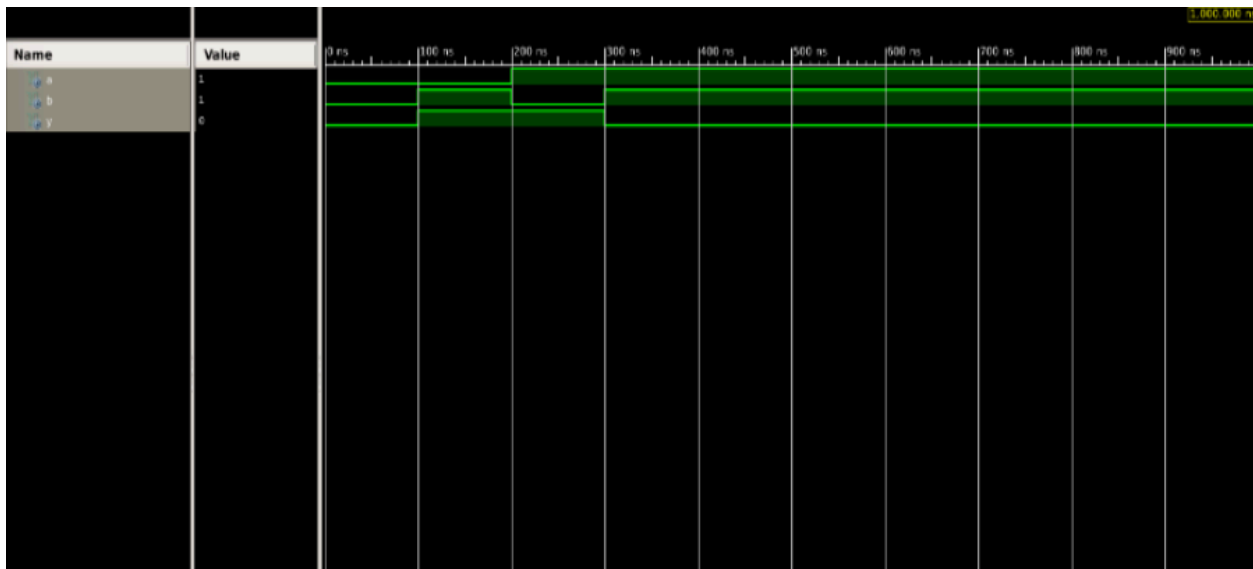


Figure 10: Simulation waveform of the XOR gate.

7. Half Adder Design

A Half Adder is a combinational circuit that adds two one-bit binary numbers.

It has two inputs:

- A
- B

It has two outputs:

- Sum
- Carry

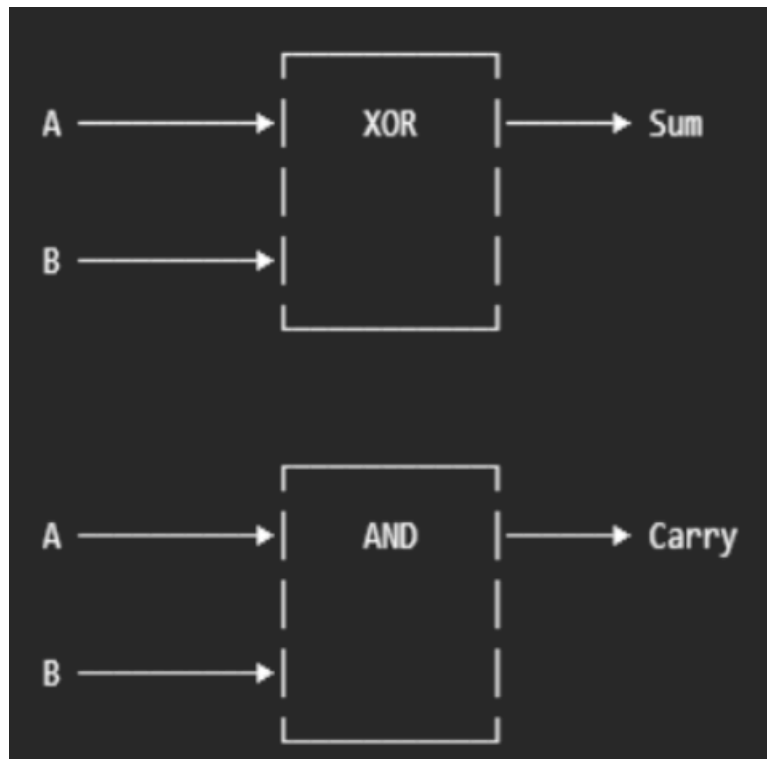
The Boolean expressions are:

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

The Half Adder was designed using the XOR and AND components created earlier.

Block Diagram



Half Adder Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

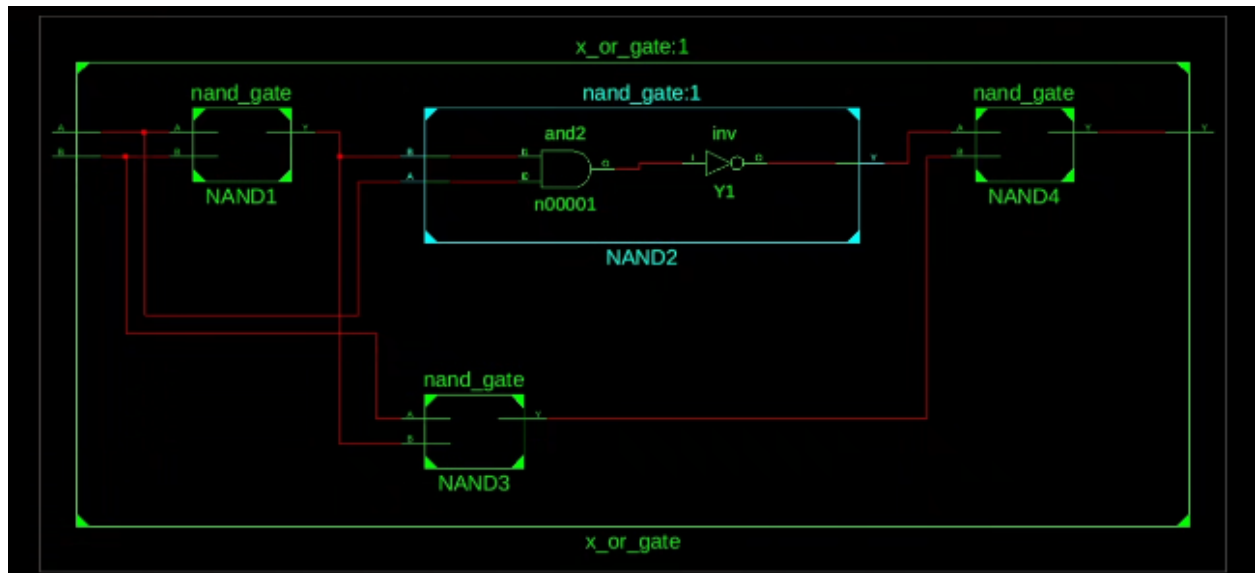


Figure 11: RTL schematic of the Half Adder.

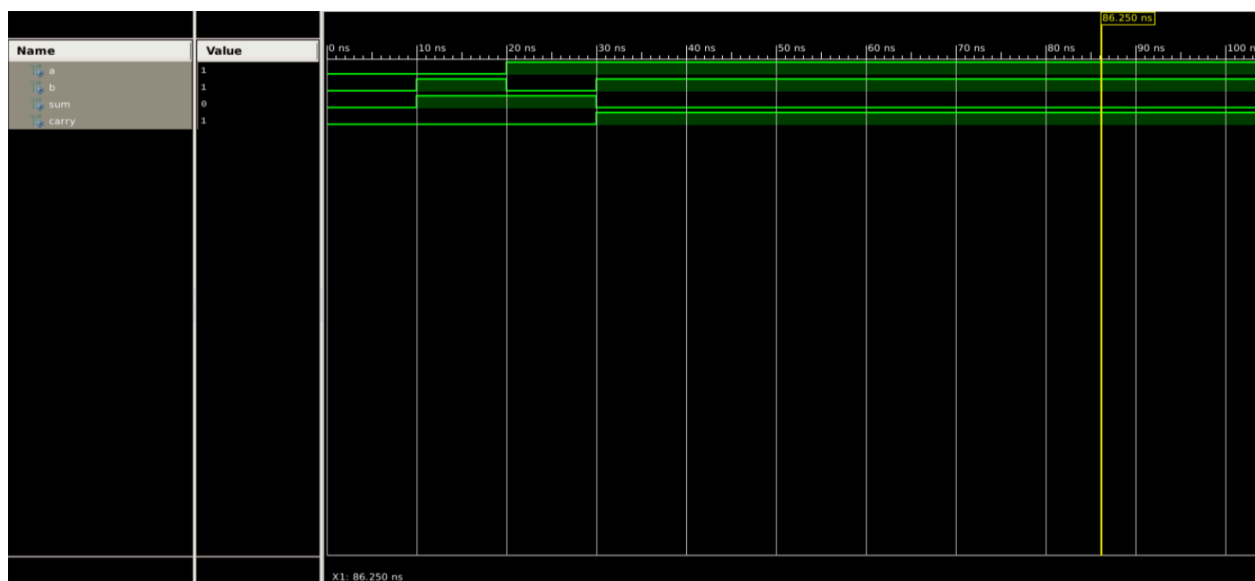


Figure 12: Simulation waveform of the Half Adder.

8. Full Adder Design

A Full Adder adds three one-bit binary values.

Inputs:

- A
- B
- Cin

Outputs:

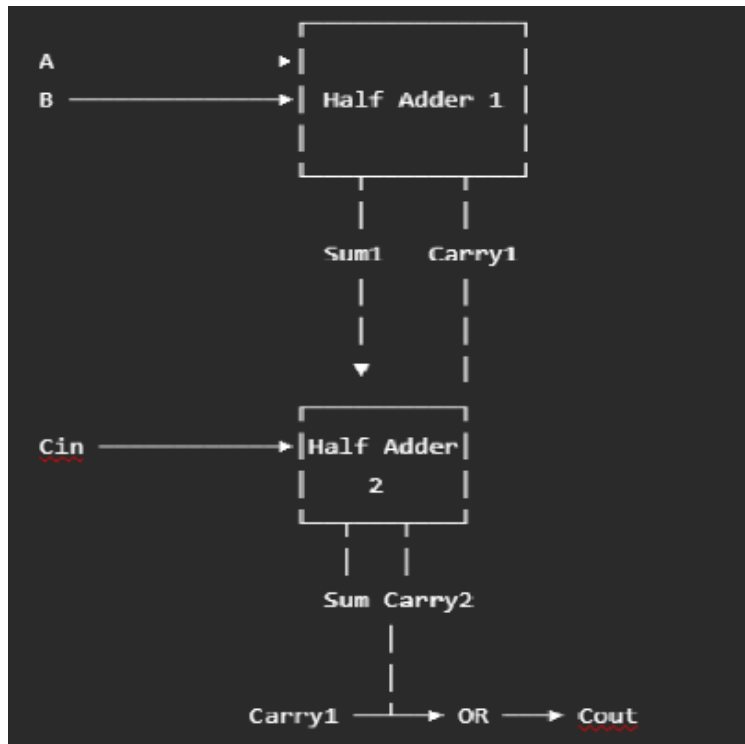
- Sum
- Cout

A Full Adder can be built using:

- Two Half Adders
- One OR gate

The first Half Adder adds A and B. The second Half Adder adds the first Half Adder's Sum with Cin. Finally, the two carry outputs are combined using an OR gate.

8.1 Full Adder Block Diagram



The Full Adder is arranged using two Half Adders and one OR gate.

9. Full Adder Logic

The first Half Adder produces:

$$\text{Sum1} = A \oplus B$$

$$\text{Carry1} = A \cdot B$$

The second Half Adder produces:

$$\text{Sum} = \text{Sum1} \oplus \text{Cin}$$

$$\text{Carry2} = \text{Sum1} \cdot \text{Cin}$$

Finally:

$$\text{Cout} = \text{Carry1} + \text{Carry2}$$

Therefore:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + (A \oplus B)\text{Cin}$$

10. Full Adder Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The simulation results should match this truth table for all eight possible input combinations.

11. RTL Schematic of the Full Adder

The final RTL schematic should show the hierarchical structure of the Full Adder.

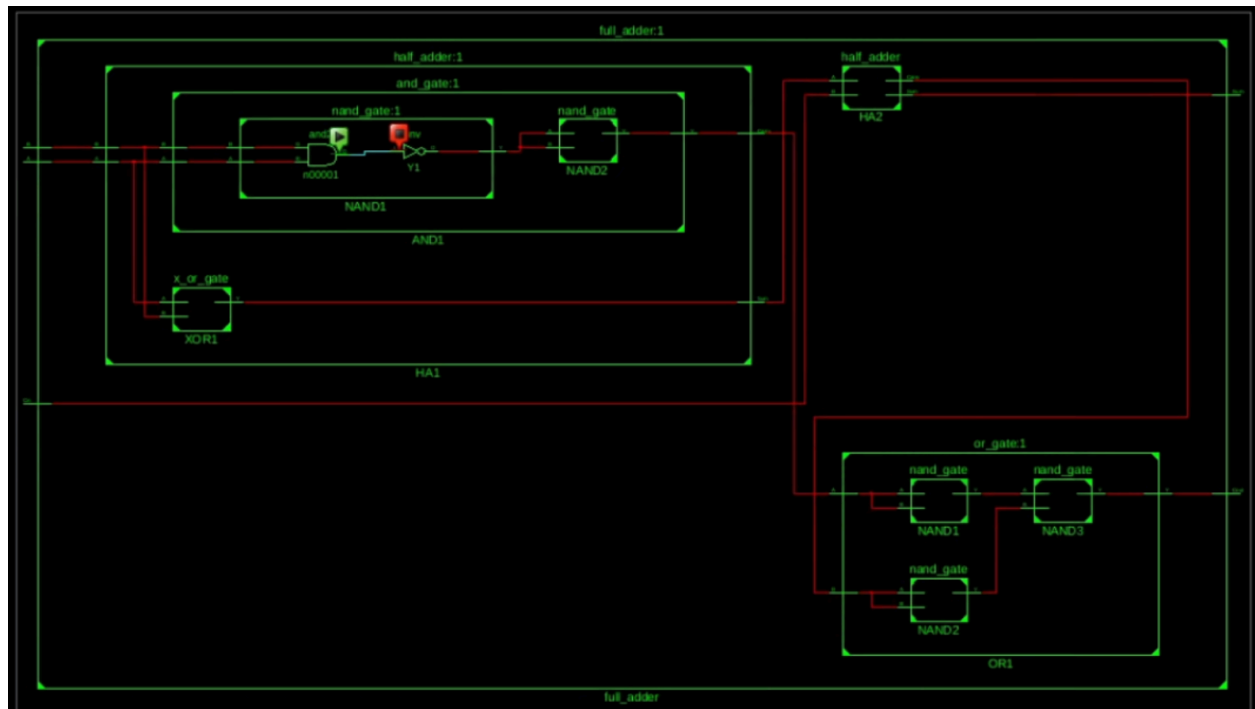


Figure 13: RTL schematic of the complete Full Adder.

This is an important screenshot in the report, so the RTL schematic should be clearly visible.

12. Simulation

The complete Full Adder was simulated using the ISim simulator in Xilinx ISE.

The testbench applies all eight possible combinations of the three inputs:

Input Combination	Decimal Value
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Each input combination is applied for a fixed amount of time. This makes it possible to observe the related Sum and Cout outputs.

Full Adder Testbench

The testbench checks the following input combinations:

$A = 0, B = 0, C_{in} = 0$

$A = 0, B = 0, C_{in} = 1$

$A = 0, B = 1, C_{in} = 0$

$A = 0, B = 1, C_{in} = 1$

$A = 1, B = 0, C_{in} = 0$

$A = 1, B = 0, C_{in} = 1$

$A = 1, B = 1, C_{in} = 0$

$A = 1, B = 1, C_{in} = 1$

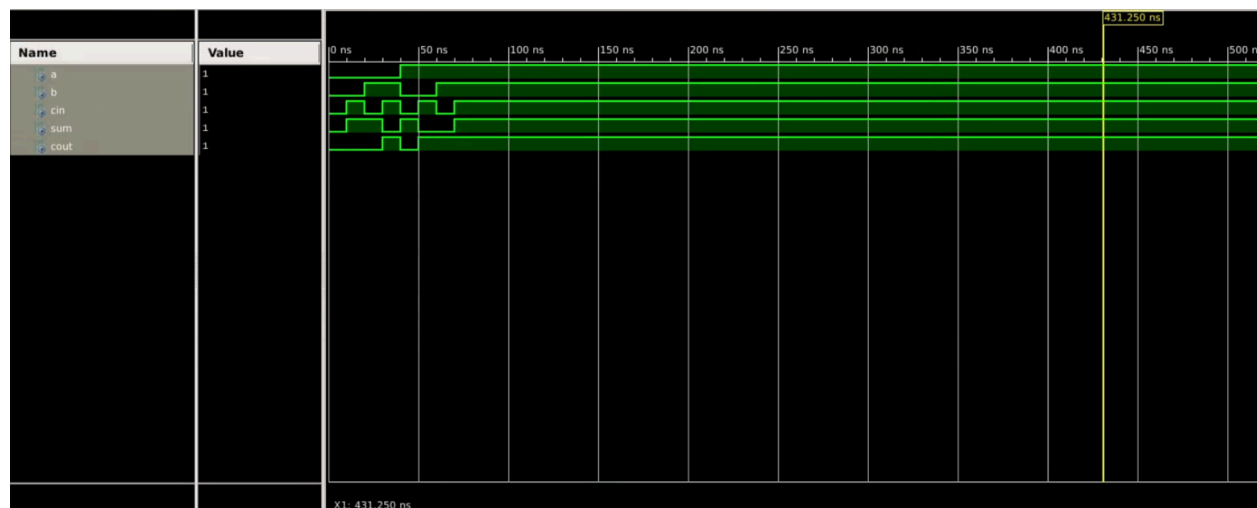


Figure 14: Simulation waveform showing all eight input combinations and their corresponding Sum and Cout outputs.

The waveform screenshot should clearly show these signals:

- A
- B
- C_{in}
- Sum
- Cout

It is better to zoom the waveform so that all eight input combinations can be seen clearly.

13. Simulation Result Analysis

The simulation confirms that the designed Full Adder works correctly.

Case 1

$$A = 0$$

$$B = 0$$

$$C_{in} = 0$$

$$\text{Sum} = 0$$

$$C_{out} = 0$$

Case 2

$$A = 0$$

$$B = 1$$

$$C_{in} = 1$$

$$\text{Sum} = 0$$

$$C_{out} = 1$$

This represents:

$$0 + 1 + 1 = 10_2$$

Therefore:

$$\text{Sum} = 0$$

$$C_{out} = 1$$

Case 3

$$A = 1$$

$$B = 1$$

$$C_{in} = 1$$

$$\text{Sum} = 1$$

$$C_{out} = 1$$

This represents:

$$1 + 1 + 1 = 11_2$$

Therefore:

$$\text{Sum} = 1$$

$$C_{out} = 1$$

The simulation output matches the theoretical Full Adder truth table.

14. GitHub Repository

All VHDL source codes, testbenches, and related project files should be uploaded to a GitHub repository.

GitHub Repository Link: <https://github.com/Rifah03/VLSI-ISE-14.7>

15. Conclusion

In this experiment, a complete Full Adder circuit was designed and simulated using VHDL and Xilinx ISE. A hierarchical structural method was followed.

First, a NAND gate was implemented. Since NAND is a universal logic gate, it was then used to create other logic gates such as AND, OR, NOR, and XOR. These gates were then used to build a Half Adder, which was later used to build the final Full Adder.

The Full Adder was built using two Half Adders and one OR gate. The final circuit takes three binary inputs (A, B, and Cin) and produces two outputs, Sum and Cout.

The circuit was simulated using all eight possible combinations of the three input signals. The simulation results matched the theoretical Full Adder truth table. Therefore, the design was successfully verified.

This experiment also showed how structural VHDL modeling, component instantiation, hierarchical design, and simulation can be used to create complex digital circuits from smaller reusable components.